

Plano de Projeto: P2P com Balanceamento de Carga Dinâmico

Visão Geral do Projeto

O objetivo deste projeto é desenvolver um sistema distribuído autônomo que demonstre o conceito de balanceamento de carga horizontal através de uma arquitetura P2P. Cada nó "Master" gerencia seu próprio conjunto de nós "Worker" (uma *Farm*). Os Masters monitoram sua carga de trabalho e, ao atingir um limiar de saturação, negociam dinamicamente o empréstimo de Workers de um Master vizinho para lidar com o excesso de requisições. A comunicação e a negociação entre os Masters devem seguir um protocolo de consenso, garantindo que a transferência de recursos seja coordenada e acordada.

O principal desafio é garantir a autonomia entre os sistemas desenvolvidos por diferentes equipes, permitindo que eles se interconectam e colaborem sem conhecimento prévio da implementação interna um do outro, apenas do protocolo de comunicação.

Objetivos

- **O1. Implementar a Arquitetura P2P:** Criar um nó Master capaz de gerenciar (iniciar, parar, monitorar) um conjunto de Workers (Farm).
- **O2. Simular Carga de Trabalho:** Desenvolver um mecanismo para simular requisições chegando a um Master, permitindo o monitoramento da carga.
- **O3. Implementar o Monitoramento de Saturação:** O Master deve ser capaz de identificar quando o número de requisições excede um limiar (threshold) pré-definido.
- **O4. Desenvolver o Protocolo de Conversa Consensual:** Projetar e implementar um protocolo robusto para que um Master saturado possa solicitar ajuda, um Master vizinho possa responder, e ambos possam coordenar o redirecionamento de Workers.
- **O5. Implementar o Redirecionamento Dinâmico de Workers:** Um Master vizinho deve ser capaz de instruir um ou mais de seus Workers a se reportarem temporariamente ao Master saturado.
- **O6. Garantir a Autonomia e Interoperabilidade:** O sistema deve funcionar em conjunto com o sistema de outra equipe, comunicando-se exclusivamente através do protocolo definido.

3. Arquitetura Proposta

O sistema será composto por três entidades principais:

1. Nó Master:

- Recebe requisições de clientes (simuladas).
- Mantém uma lista de Workers em sua Farm.
- Distribui requisições entre seus Workers.
- Monitora constantemente o número de requisições pendentes.
- Inicia o "Protocolo de Conversa Consensual" quando requisições > threshold.
- Gerencia Workers "emprestados" de outros Masters.

2. Nó Worker:

- Recebe uma tarefa do seu Master atual e a processa (ex: um cálculo simples, um sleep para simular trabalho).
- Deve ser capaz de se desconectar do seu Master original e se conectar a um novo Master quando instruído.

3. Protocolo de Comunicação (Mensageria):

- A espinha dorsal do projeto. Define as regras para a negociação.
- Será baseado em trocas de mensagens, por exemplo, via API REST, gRPC ou Sockets customizados com formato JSON.

Considerações de Implementação

- **Message Delimiter:** Em um stream de TCP, você precisa de uma forma de saber onde uma mensagem JSON termina e a próxima começa. A abordagem mais simples é enviar um caractere de nova linha (\n) após cada objeto JSON. O receptor lê o socket até encontrar um \n, e então processa o que recebeu como um JSON completo.
- **Escuta Contínua:** Cada Master e Worker deve rodar em um loop infinito, escutando por novas conexões (se for servidor) ou novas mensagens em conexões existentes.
- **Threads/AsyncIO:** É fundamental usar concorrência (threads ou programação assíncrona) para que um Master possa se comunicar com múltiplos outros Masters e Workers simultaneamente sem bloquear seu processo principal.
- **Protocolos de Comunicação devem seguir o padrão estabelecido em sala.**

PAYLOAD OFICIAL:

[1] HEARTBEAT - Worker A1 - Master A - Verificação de Atividade.

[1.1] Worker A1 → Master A - Worker A1 pergunta ao Servidor B se ele está ativo.

PAYLOAD:

```
{  
  "SERVER_UUID": "Master_A",  
  "TASK": "HEARTBEAT"  
}
```

[1.2] Master A → Worker A1 - Servidor B responde que está ativo.

PAYLOAD:

```
{  
  "SERVER_UUID": "Master_A",  
  "TASK": "HEARTBEAT",  
  "RESPONSE": "ALIVE"  
}
```

Sprint: Implementação do Mecanismo de Heartbeat (Prazo Entrega: Ver Atividade no Sala Online)

Estabelecer a comunicação base entre o Worker e o Master, garantindo que o Worker consiga verificar se o seu "mestre" está ativo através de trocas de mensagens JSON via TCP.

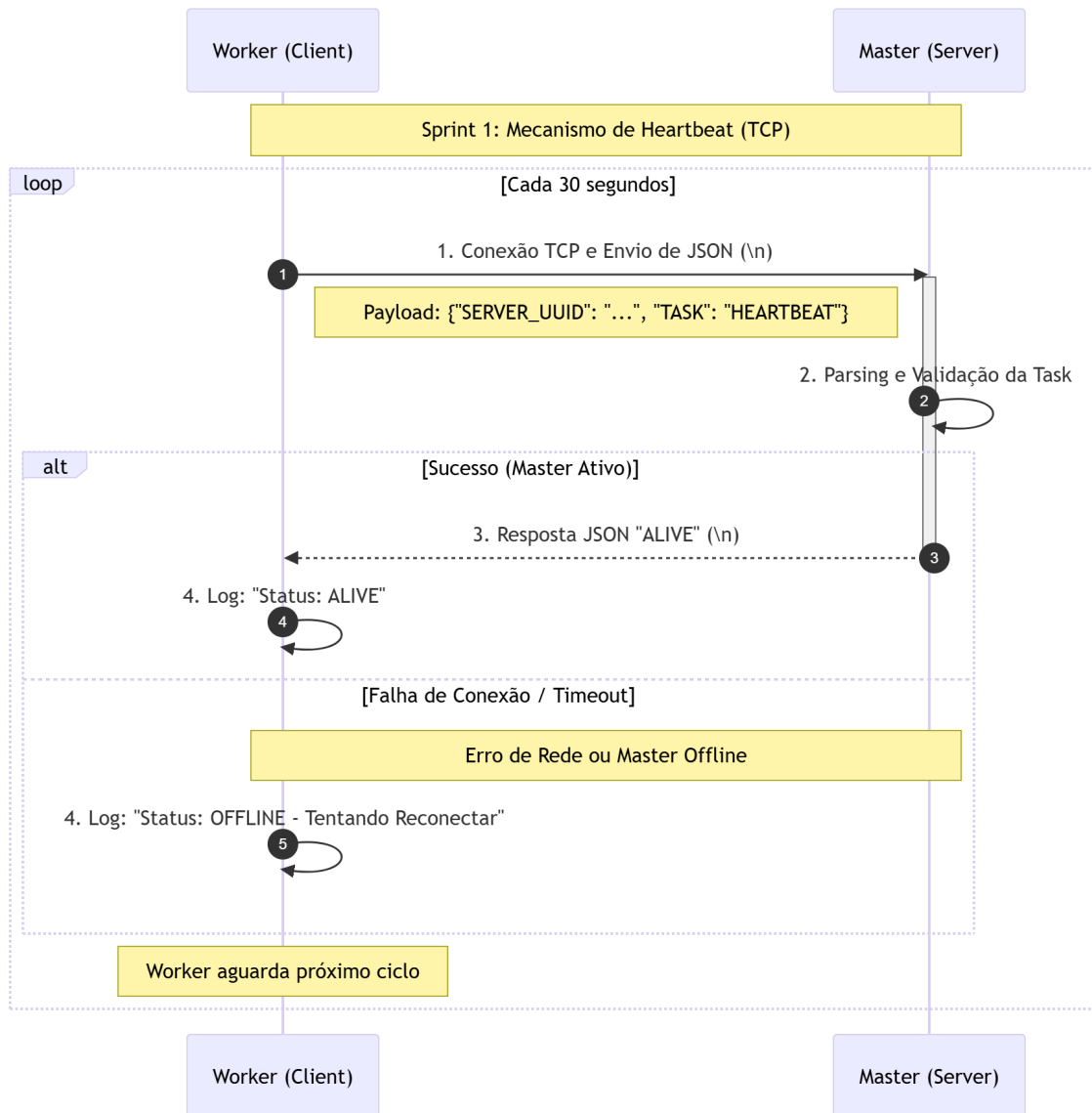
1. Backlog de Tarefas (To-Do)

- **Tarefa 01: Infraestrutura de Comunicação TCP**
 - Configurar o Master para atuar como servidor, escutando em uma porta definida.
 - Configurar o Worker para atuar como cliente, iniciando a conexão.
 - Padrão de Mensagem: Implementar o delimitador de nova linha (`\n`) ao final de cada JSON para garantir que o receptor identifique o fim da mensagem no stream TCP.
- **Tarefa 02: Lógica de Requisição (Worker → Master)**
 - Desenvolver a função no Worker para disparar o payload de verificação.
 - Payload de Envio: `{"SERVER_UUID": "...", "TASK": "HEARTBEAT"}`.
- **Tarefa 03: Lógica de Resposta (Master → Worker)**
 - Implementar no Master a capacidade de interpretar a tarefa "HEARTBEAT" e devolver a confirmação imediata.
 - Payload de Resposta: `{"SERVER_UUID": "...", "TASK": "HEARTBEAT", "RESPONSE": "ALIVE"}`.
- **Tarefa 04: Concorrência e Resiliência**
 - Utilizar Threads ou AsyncIO no Master para que o atendimento de um Heartbeat não bloqueie outras operações (como o processamento de tarefas reais).
 - Criar um loop no Worker para repetir essa verificação em intervalos regulares (ex: a cada 10 segundos).

2. Definição de "Pronto" (DoD)

A entrega será considerada concluída quando:

1. O Worker conseguir abrir uma conexão TCP com o Master.
2. O Master receber o JSON, realizar o *parsing* e identificar o comando de Heartbeat.
3. O Worker receber a confirmação "ALIVE" e conseguir imprimir isso no log.
4. A conexão for mantida ou reestabelecida corretamente sem travar os processos.



Sprint 2: Comunicação de Tarefas e Apresentação de Workers

Implementar o fluxo completo de ciclo de vida de uma tarefa, desde a apresentação do Worker (identificando sua origem) até o processamento e a confirmação final de recebimento do status pelo Master.

1. Backlog de Tarefas (To-Do)

- **Tarefa 01: Lógica de Apresentação e Identificação (Worker → Master)**
 - Implementar no Worker a capacidade de se apresentar ao Master enviando seu UUID.
 - *Diferencial de Origem: O Worker deve ser capaz de enviar o payload de "Emprestado" caso esteja vinculado a um Master original diferente.*
 - Payloads (2.1 e 2.1b): `{"WORKER": "ALIVE", "WORKER_UUID": "..."} ou {"WORKER": "ALIVE", "WORKER_UUID": "...", "SERVER_UUID": "...}`.
- **Tarefa 02: Distribuição de Carga e Gestão de Fila (Master → Worker)**
 - Configurar o Master para gerenciar uma fila (queue) de tarefas pendentes.
 - Ao receber um pedido de tarefa, o Master deve verificar a fila:
 - Com Tarefa (Payload 2.2): Enviar `{"TASK": "QUERY", "USER": "..."}.`
 - Fila Vazia (Payload 2.3): Enviar `{"TASK": "NO_TASK"}`.
- **Tarefa 03: Simulação de Processamento e Relatório de Status (Worker → Master)**
 - Desenvolver no Worker o "executor": ao receber uma `QUERY`, ele deve simular um processamento (ex: `sleep` aleatório ou cálculo).
 - Após o trabalho, o Worker deve reportar o resultado.
 - Payload (2.4): `{"STATUS": "OK | NOK", "TASK": "QUERY", "WORKER_UUID": "..."}.`
- **Tarefa 04: Mecanismo de Confirmação (ACK) e Persistência (Master → Worker)**
 - Implementar no Master o recebimento do status e a resposta de confirmação imediata para liberar o Worker.
 - Payload (2.5): `{"STATUS": "ACK"}`.
 - Garantir que o Master registre (log) qual Worker (local ou emprestado) concluiu qual tarefa.

2. Definição de "Pronto" (DoD)

A entrega será considerada concluída quando:

1. O Worker realizar o "handshake" de apresentação com sucesso (enviando UUID).
2. O Master distribuir uma tarefa real da fila ou informar corretamente que não há tarefas.
3. O Worker processar a tarefa e o Master receber o status `OK` ou `NOK`.
4. O Worker receber o `ACK` final do Master, fechando o ciclo de comunicação sem erros de parsing ou perda de mensagens no stream TCP.
5. O sistema tratar corretamente a presença (ou ausência) do campo `SERVER_UUID` no

payload de apresentação.

Definição do Payload Padrão

Para que o sistema de uma equipa "A" fale com o sistema da equipa "B", todos os pacotes devem seguir rigorosamente esta estrutura JSON, terminando sempre com o caractere `\n`.

1. Solicitação de Tarefa (Worker → Master)

- Campos obrigatórios: `WORKER`, `WORKER_UUID`.
- Campo opcional: `SERVER_UUID` (usado apenas se o worker for "emprestado").

```
{  
  "WORKER": "ALIVE",  
  "WORKER_UUID": "string",  
  "SERVER_UUID": "string"  
}
```

2. Entrega de Tarefa (Master → Worker)

Caso haja tarefa:

```
{  
  "TASK": "QUERY",  
  "USER": "string"  
}
```

Caso não haja tarefa:

```
{  
  "TASK": "NO_TASK"  
}
```

3. Reporte de Status (Worker → Master)

Campos obrigatórios: `STATUS` (valores: "OK" ou "NOK"), `TASK`, `WORKER_UUID`.

```
{  
  "STATUS": "OK",  
  "TASK": "QUERY",  
  "WORKER_UUID": "string"  
}
```

4. Confirmação Final (Master → Worker)

Campo único: `STATUS` (valor fixo: "ACK").

```
{  
  "STATUS": "ACK",  
  "WORKER_UUID": "string"  
}
```

Notas de Implementação:

1. **Strict Parsing:** Os Masters devem ser programados para ignorar campos desconhecidos no JSON (para permitir extensões futuras), mas devem falhar se os campos obrigatórios acima estiverem ausentes.
2. **Case Sensitivity:** Todos os valores de controle (`ALIVE`, `QUERY`, `NO_TASK`, `OK`, `NOK`, `ACK`) devem ser tratados em CAIXA ALTA.
3. **Timeout:** O Worker deve aguardar a resposta do Master por no máximo 5 segundos antes de considerar a conexão perdida e tentar uma nova conexão.

ID	Cenário de Teste	Ação do Worker (JSON)	Resposta Esperada do Master	Critério de Sucesso
CT01	Apresentação Worker Local	<code>{"WORKER": "ALIVE", "WORKER_UUID": "W-123"}</code>	<code>{"TASK": "QUERY", "USER": "Michel"}</code>	Master identifica Worker local e entrega uma tarefa da fila.
CT02	Apresentação Worker Emprestado	<code>{"WORKER": "ALIVE", "WORKER_UUID": "W-999", "SERVER_UUID": "Master-B"}</code>	<code>{"TASK": "QUERY", "USER": "Julia"}</code>	Master reconhece que o Worker pertence a outro nó, mas atribui tarefa normalmente.

CT03	Fila de Tarefas Vazia	<code>{"WORKER": "ALIVE", "WORKER_UUID": "W-123"}</code>	<code>{"TASK": "NO_TASK"}</code>	O Master responde corretamente quando não há trabalho pendente.
CT04	Reporte de Sucesso	<code>{"STATUS": "OK", "TASK": "QUERY", "WORKER_UUID": "W-123"}</code>	<code>{"STATUS": "ACK"}</code>	O Master processa o sucesso e liberta o Worker com um ACK.
CT05	Reporte de Falha	<code>{"STATUS": "NOK", "TASK": "QUERY", "WORKER_UUID": "W-123"}</code>	<code>{"STATUS": "ACK"}</code>	O Master registra a falha, mas ainda assim envia o ACK para confirmar o recebimento do status.

Sprint 03 - Protocolo de Negociação Master-to-Master e Redirecionamento Dinâmico de Workers

Objetivo da Sprint

Implementar a camada de comunicação P2P entre Masters, permitindo que um Master saturado negocie e receba, de forma autônoma e consensual, Workers emprestados de um Master vizinho. A sprint cobre o ciclo completo: pedido de ajuda, análise/resposta, comando de redirecionamento, registro temporário do Worker no Master saturado e devolução do Worker ao seu Master de origem quando a carga normalizar.

Esta entrega cumpre integralmente os objetivos O4 (Protocolo de Conversa Consensual), O5 (Redirecionamento Dinâmico de Workers) e O6 (Autonomia e Interoperabilidade) definidos no plano geral do projeto.

Pré-requisitos

4. Sprint 01 concluída: mecanismo de Heartbeat (Worker ↔ Master) operacional.
5. Sprint 02 concluída: ciclo de tarefas (apresentação, distribuição, status, ACK) operacional, incluindo suporte ao campo SERVER_UUID para Workers emprestados.
6. Cada Master deve possuir um identificador único (master_id) e endereço de socket (ip:porta) conhecido pelos Masters vizinhos.

1. Estrutura Padrão da Mensagem (JSON)

Toda comunicação Master-to-Master enviada pelo socket seguirá esta estrutura. O campo "type" substitui a URL da API REST e identifica a operação requisitada. O campo "request_id" é um UUID único que correlaciona requisição e resposta, mesmo em conexões concorrentes. Toda mensagem é finalizada com o caractere \n, conforme padrão estabelecido nas sprints anteriores.

Estrutura genérica:

```
{
  "type": "TIPO_DA_MENSAGEM",
  "request_id": "uuid_unico_para_rastreo",
  "payload": {
    // ... dados específicos da mensagem
  }
}
```

2. Passo a Passo do Fluxo via Sockets

O fluxo abaixo descreve o ciclo completo de empréstimo e devolução de um Worker entre dois Masters. Todas as mensagens devem trafegar pela mesma conexão TCP previamente estabelecida entre os Masters (salvo nos casos explicitamente indicados, em que o Worker abre uma nova conexão com o Master de destino).

2.1. Pedido de Ajuda (request_help)

O Master A, ao detectar saturação (requisições pendentes acima do threshold definido), abre uma conexão de socket com o Master B. Uma vez conectado, envia a seguinte mensagem JSON:

De: Master A Para: Master B

```
{
  "type": "request_help",
  "request_id": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
  "payload": {
    "master_id": "A",
    "current_load": 150,
    "capacity": 100,
    "workers_needed": 2
  }
}
```

2.2. Análise e Resposta

O Master B recebe a mensagem, avalia sua própria carga e a disponibilidade de Workers ociosos, e responde pela mesma conexão de socket. Há dois desfechos possíveis:

Resposta de Sucesso (response_accepted) — Master B → Master A:

```
{
  "type": "response_accepted",
  "request_id": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
  "payload": {
    "workers_offered": 2,
    "worker_details": [
      { "id": "B1", "address": "ip:port_worker_b1" },
      { "id": "B2", "address": "ip:port_worker_b2" }
    ]
  }
}
```

Resposta de Falha (response_rejected) — Master B → Master A:

```
{
  "type": "response_rejected",
  "request_id": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
  "payload": {
    "reason": "high_load"
  }
}
```

Observação: o request_id da resposta deve ser idêntico ao da requisição original, permitindo que o Master A correlacione a resposta mesmo em cenários com múltiplos pedidos simultâneos. Possíveis valores para o campo "reason" incluem: high_load, no_workers_available e refused.

2.3. Comando de Redirecionamento (command_redirect)

Após enviar response_accepted, o Master B comunica-se com cada um dos Workers ofertados (pela conexão de socket que ele já mantém com eles, conforme Sprint 02) e ordena o redirecionamento para o Master A. Esta mensagem possui um novo request_id, pois pertence a um fluxo distinto (Master ↔ Worker).

De: Master B Para: Worker B1

```
{
  "type": "command_redirect",
  "request_id": "f0e9d8c7-b6a5-4321-fedc-ba9876543210",
  "payload": {
    "new_master_address": "ip_master_A:port"
  }
}
```

2.4. Registro do Worker Temporário (register_temporary_worker)

Ao receber command_redirect, o Worker B1 encerra graciosamente sua conexão com o Master B (finalizando qualquer tarefa em execução, conforme Sprint 02) e abre uma nova conexão de socket com o Master A. Imediatamente após conectar, o Worker apresenta-se enviando seu identificador e o endereço de seu Master de origem, para que o Master A saiba que se trata de um Worker emprestado:

De: Worker B1 Para: Master A

```
{
  "type": "register_temporary_worker",
  "request_id": "c1b2a3d4-e5f6-g7h8-i9j0-k1l2m3n4o5p6",
  "payload": {
    "worker_id": "B1",
    "original_master_address": "ip_master_B:port"
  }
}
```

A partir desse momento, o Worker B1 passa a operar sob o controle do Master A, obedecendo ao mesmo ciclo de tarefas definido na Sprint 02 (apresentação ALIVE com SERVER_UUID, QUERY/NO_TASK, reporte de STATUS e recebimento de ACK).

2.5. Devolução do Worker

Quando a carga do Master A normaliza (requisições pendentes abaixo de um threshold de liberação, tipicamente menor do que o threshold de saturação para evitar oscilações — efeito histerese), o processo de devolução ocorre em duas etapas, descritas a seguir.

2.5.a) Comando para o Worker retornar (command_release)

Master A instrui o Worker B1 a encerrar a conexão e retornar ao seu Master original:

De: Master A Para: Worker B1

```
{
  "type": "command_release",
  "request_id": "z9y8x7w6-v5u4-t3s2-r1q0-p9o8n7m6l5k4",
  "payload": {
    "original_master_address": "ip_master_B:port"
  }
}
```

2.5.b) Notificação de devolução (notify_worker_returned)

Em paralelo, Master A notifica Master B (pela conexão Master-to-Master original) que o Worker foi liberado, permitindo que Master B atualize sua Farm e volte a aceitar o Worker:

De: Master A Para: Master B

```
{
  "type": "notify_worker_returned",
  "request_id": "m1n2b3v4-c5x6-z7a8-s9d0-f1g2h3j4k5l6",
  "payload": {
    "worker_id": "B1"
  }
}
```

Após receber command_release, o Worker B1 reconecta-se ao Master B usando o protocolo padrão de apresentação (Sprint 02). A conexão entre Master A e Master B pode ser fechada ou mantida em um pool de conexões reutilizável para futuras negociações.

3. Resumo dos Tipos de Mensagem

Tabela consolidada de todos os "type" definidos nesta sprint:

type	De → Para	Quando ocorre	Finalidade
request_help	Master A → Master B	Carga > threshold	Solicita Workers emprestados a um Master vizinho.
response_accepted	Master B → Master A	Master B tem capacidade ociosa	Aceita o pedido e informa quais Workers serão emprestados.
response_rejected	Master B → Master A	Master B sem capacidade	Recusa o pedido informando o motivo (reason).
command_redirect	Master B → Worker B1	Após response_accepted	Ordena o Worker a se reportar ao Master saturado.
register_temporary_worker	Worker B1 → Master A	Após reconectar	Apresenta-se ao novo Master indicando o Master de origem.
command_release	Master A → Worker B1	Carga normalizou	Libera o Worker para retornar ao Master original.
notify_worker_returned	Master A → Master B	Após command_release	Notifica o Master B que o Worker foi devolvido.

4. Backlog de Tarefas (To-Do)

Tarefa 01 — Conexão TCP entre Masters

7. Implementar no Master a capacidade de atuar simultaneamente como servidor (escutando conexões de Workers e de outros Masters) e como cliente (abrindo conexões com Masters vizinhos).
8. Manter um diretório de Masters vizinhos contendo master_id e endereço (ip:porta).
9. Reaproveitar o delimitador \n já adotado nas Sprints 01 e 02 para enquadramento das mensagens JSON.

Tarefa 02 — Detecção de Saturação

10. Definir e parametrizar o threshold de saturação (ex.: capacity = 100 requisições pendentes).
11. Definir um threshold de liberação (ex.: 60% da capacidade) para acionar a devolução, evitando oscilações (efeito histerese).
12. Disparar o envio de request_help quando current_load > capacity, calculando workers_needed proporcionalmente ao excedente.

Tarefa 03 — Protocolo de Negociação (request_help / response)

13. Implementar emissão de request_help com geração de UUID v4 para o request_id.
14. Implementar receptor no Master que avalia carga atual, número de Workers ociosos e responde com response_accepted ou response_rejected.
15. Garantir que o request_id da resposta seja idêntico ao da requisição original.
16. Implementar timeout de 5 segundos no Master solicitante; após o timeout, considerar o pedido como recusado e tentar próximo vizinho.

Tarefa 04 — Redirecionamento de Workers

17. Implementar emissão de `command_redirect` do Master ofertante para cada Worker selecionado.
18. Atualizar o Worker para tratar `command_redirect`: encerrar a conexão atual graciosamente (sem perder tarefa em execução), conectar ao novo endereço e enviar `register_temporary_worker`.
19. Atualizar o Master receptor para tratar `register_temporary_worker`, registrando o Worker como "emprestado" e seu Master de origem.
20. A partir do registro, o Worker emprestado deve operar pelo protocolo da Sprint 02 enviando ALIVE com o campo `SERVER_UUID` preenchido com o Master de origem.

Tarefa 05 — Devolução do Worker

21. Implementar lógica que monitora o retorno da carga abaixo do threshold de liberação no Master A.
22. Implementar emissão de `command_release` do Master A para o Worker emprestado.
23. Implementar emissão de `notify_worker_returned` do Master A para o Master B na conexão Master-to-Master.
24. Garantir que o Worker se reconecte ao Master original e volte a operar normalmente, sem perda de estado.

Tarefa 06 — Concorrência e Resiliência

25. Utilizar Threads ou AsyncIO para que o Master atenda Workers próprios, Workers emprestados, conexões com Masters vizinhos e simulação de carga simultaneamente, sem bloqueio.
26. Tratar desconexões inesperadas: se um Worker emprestado perder conexão com o Master A, ele deve tentar voltar ao Master B; se um Master vizinho cair durante a negociação, o solicitante deve liberar o `request_id` e seguir o fluxo.
27. Toda mensagem recebida com type desconhecido deve ser logada e ignorada, sem derrubar o processo (compatibilidade com extensões futuras).

Tarefa 07 — Logs e Observabilidade

28. Registrar em log toda emissão e recebimento de mensagens Master-to-Master com seu `request_id`, type e timestamp.
29. Manter contador de Workers locais e emprestados por Master, exibindo o estado a cada mudança.
30. Registrar o ciclo de vida completo de cada Worker emprestado: empréstimo, registro, tarefas executadas e devolução.

5. Definição de "Pronto" (DoD)

A entrega da Sprint 03 será considerada concluída quando:

2. Um Master saturado consegue abrir conexão TCP com um Master vizinho e enviar `request_help` corretamente formatado.
3. O Master vizinho processa a requisição e responde com `response_accepted` ou `response_rejected`, mantendo o mesmo `request_id`.

4. Após `response_accepted`, o Master vizinho envia `command_redirect` aos Workers acordados, e estes encerram a conexão original e se reconectam ao Master saturado.
5. Os Workers emprestados executam `register_temporary_worker` e passam a receber tarefas do Master saturado, identificando-se como emprestados (campo `SERVER_UUID` na apresentação).
6. Quando a carga normaliza, o Master saturado emite `command_release` ao Worker e `notify_worker_returned` ao Master de origem; o Worker reconecta-se com sucesso ao Master original.
7. O sistema funciona em interoperabilidade com a implementação de outra equipe, comunicando-se exclusivamente pelos payloads definidos.
8. O parsing tolera campos desconhecidos e falha de forma controlada (com log) quando campos obrigatórios estão ausentes.
9. Não há vazamento de threads, conexões TCP penduradas ou perda de mensagens no stream após o ciclo completo.

6. Casos de Teste

ID	Cenário	Ação / Mensagem Disparada	Resultado Esperado
CT01	Pedido de ajuda aceito	Master A envia <code>request_help</code> com <code>workers_needed = 2</code> quando há 2 Workers ociosos no Master B.	Master B responde <code>response_accepted</code> com <code>worker_details</code> contendo 2 workers; em seguida envia <code>command_redirect</code> a cada um.
CT02	Pedido de ajuda recusado	Master A envia <code>request_help</code> para um Master B com carga alta.	Master B responde <code>response_rejected</code> com <code>reason = "high_load"</code> ; nenhum <code>command_redirect</code> é emitido.
CT03	Correlação de <code>request_id</code>	Master A envia 2 <code>request_help</code> concorrentes para Masters distintos.	Cada resposta retorna com o <code>request_id</code> idêntico ao da requisição original e é corretamente correlacionada.
CT04	Registro de Worker emprestado	Worker B1, após <code>command_redirect</code> , conecta no Master A e envia <code>register_temporary_worker</code> .	Master A registra o Worker como emprestado; nas tarefas seguintes (Sprint 02) o Worker envia <code>ALIVE</code> com <code>SERVER_UUID = Master B</code> .
CT05	Tarefa em Worker emprestado	Master A possui tarefa na fila e seu Worker emprestado solicita trabalho.	Master A entrega <code>QUERY</code> ao Worker emprestado; Worker reporta <code>STATUS OK</code> ; Master A envia <code>ACK</code> e registra no log que tarefa foi executada por Worker emprestado.
CT06	Devolução do Worker	Carga do Master A cai abaixo do <code>threshold</code> de liberação.	Master A envia <code>command_release</code> ao Worker B1 e <code>notify_worker_returned</code> ao Master B; Worker B1 reconecta no Master B e volta a receber tarefas locais.

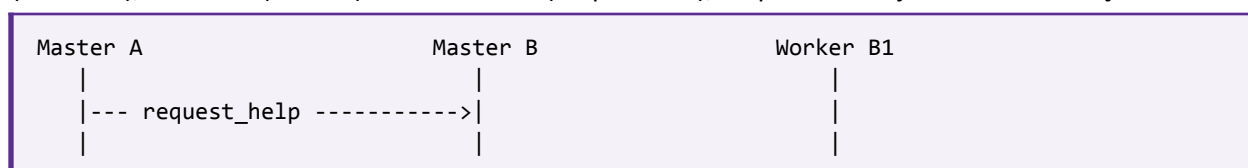
ID	Cenário	Ação / Mensagem Disparada	Resultado Esperado
CT07	Timeout de negociação	Master A envia request_help, mas Master B não responde em 5 segundos.	Master A descarta o request_id, registra o timeout no log e tenta o próximo vizinho ou aborta o pedido.
CT08	Falha do Master receptor	Master A perde a conexão com o Master B durante o empréstimo de Workers.	Workers emprestados detectam a queda do Master A e tentam reconectar ao Master B; estado consistente é restaurado.
CT09	Tipo desconhecido	Master B recebe mensagem com type não previsto.	Master B registra em log, ignora a mensagem e continua operando normalmente.

7. Notas de Implementação

31. **Strict Parsing:** campos desconhecidos no JSON devem ser ignorados (compatibilidade com extensões futuras), mas mensagens com campos obrigatórios ausentes devem falhar com log de erro, sem derrubar o processo.
32. **Case Sensitivity:** todos os valores de "type" devem ser tratados em letras minúsculas exatamente como definidos (request_help, response_accepted, response_rejected, command_redirect, register_temporary_worker, command_release, notify_worker_returned).
33. **request_id:** deve ser um UUID v4 gerado a cada nova requisição. A resposta a um request_help reutiliza o mesmo request_id; já command_redirect, register_temporary_worker, command_release e notify_worker_returned são fluxos independentes e podem ter request_id próprio.
34. **Timeout:** o Master solicitante deve aguardar a resposta por no máximo 5 segundos antes de considerar o vizinho indisponível.
35. **Histerese:** o threshold de liberação deve ser menor que o de saturação para evitar empréstimo e devolução imediatos do mesmo Worker (efeito ping-pong).
36. **Compatibilidade com Sprint 02:** uma vez registrado, o Worker emprestado opera pelo mesmo protocolo da Sprint 02, com a única diferença de incluir o campo SERVER_UUID (Master de origem) no payload de apresentação ALIVE.
37. **Pool de conexões:** recomenda-se manter as conexões Master-to-Master abertas em um pool, evitando o custo de novo handshake TCP a cada negociação.
38. **Concorrência:** use Threads ou AsyncIO; em qualquer caso, a fila de tarefas, o conjunto de Workers e o registro de Workers emprestados devem ser protegidos contra condições de corrida (locks, semáforos ou estruturas thread-safe).

8. Resumo Visual do Fluxo

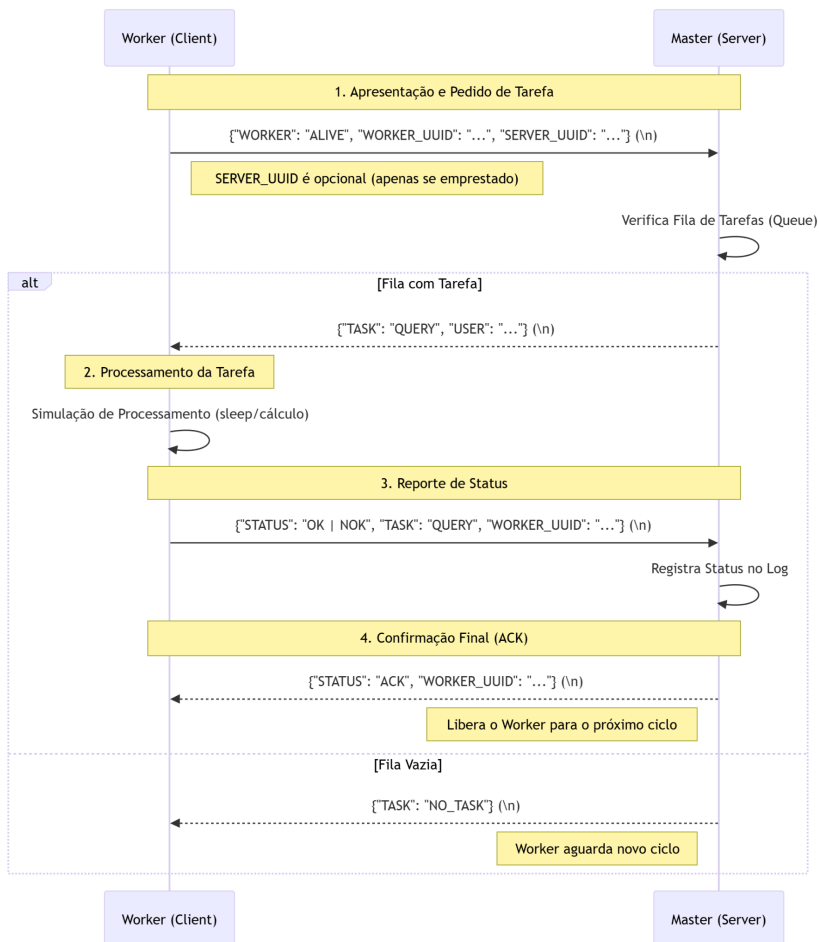
O diagrama a seguir resume, em alto nível, a sequência de mensagens trocadas entre Master A (saturado), Master B (vizinho) e o Worker B1 (emprestado), do pedido de ajuda até a devolução:



```

|<-- response_accepted -----|
|
|                                |--- command_redirect ---->|
|                                [Worker desconecta]
|
|<===== nova conexão TCP =====|
|
|<-- register_temporary_worker -----|
|
|    ... ciclo de tarefas (Sprint 02) com SERVER_UUID
|
|--- command_release ----->|
|
|--- notify_worker_returned ->|
|
|                                |<==== reconexão TCP =====|
|                                ... ciclo Sprint 02 ...

```



SPRINT 4 - APRESENTAÇÃO FINAL

APRESENTAÇÃO MATUTINO E NOTURNO: 15/06/2026 (TURMA B e UN)

APRESENTAÇÃO MATUTINO: 11/06/2025 (TURMA A)

PROVA MATUTINO E NOTURNO: 22/06/2026 (TURMA B e UN)

PROVA MATUTINO: 18/06/2026 (TURMA A)

SCHEDULE DA SIMULAÇÃO:

ATENÇÃO: A SIMULAÇÃO CONSIDERA QUE AS RNS ESTÃO DE ACORDO COM O ESPECIFICADO NO PROJETO E DEVEM ESTAR EM PLENO FUNCIONAMENTO.

A comunicação com o supervisor será por socket TCP na porta 8000 a cada 10s
Os SERVERS não devem aguardar mensagem (RECV) com retorno, apenas executar o SEND. **DASHBOARD PARA TESTE ABAIXO**

PAYLOAD:

```
{
  "server_uuid": "michel_1",
  "hostname": "michel_1.farm.local",
  "role": "master",
  "task": "performance_report",
  "timestamp": "2026-06-08T12:34:56Z",
  "message_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "payload_version": "sprint4-monitor",

  "performance": {
    "system": {
      "uptime_seconds": 12345,
      "load_average_1m": 3.20,
      "load_average_5m": 2.50,
      "cpu": {
        "usage_percent": 85.42,
        "count_logical": 8,
        "count_physical": 4
      },
    },
    "memory": {
      "total_mb": 16384,
      "available_mb": 8192,
      "percent_used": 62.18,
      "memory_used": 8000
    },
  },
}
```

```
    "disk": {
      "total_gb": 512.0,
      "free_gb": 250.0,
      "percent_used": 45.0
    }
  },
  "farm_state": {
    "workers": {
      "total_registered": 6,
      "workers_utilization": 4,
      "workers_alive": 6,
      "workers_idle": 2,
      "workers_borrowed": 1,
      "workers_received": 1,
      "workers_failed": 0,
      "workers_home": 5,
      "workers_available_capacity": 2,

      "borrowed_workers": [
        { "direction": "out", "peer_uuid": "michel_2" },
        { "direction": "in", "peer_uuid": "michel_2" }
      ]
    },
    "tasks": {
      "tasks_pending": 42,
      "tasks_running": 4,
      "tasks_completed": 150,
      "tasks_failed": 3,
      "oldest_task_age_s": 312
    }
  },
  "config_thresholds": {
    "max_task": 100,
    "warn_cpu_percent": 85,
    "warn_memory_percent": 85,
    "release_task": 60
  },
  "neighbors": [
    {
      "server_uuid": "michel_2",
      "status": "available",
      "last_heartbeat": "2026-06-08T12:34:56Z"
    }
  ]
}
```

Uso do Supervisor de Métricas do Cluster

Foi implementado um Supervisor de Métricas para monitorar, em tempo real, os projetos desenvolvidos na disciplina. Esse supervisor recebe relatórios de desempenho via TCP e apresenta as informações em um dashboard web acessível pelo navegador.

Descrição dos itens do payload:

Campo	Tipo	Descrição
server_uuid	string	Identificador único do servidor no cluster (ex: "michel_1")
hostname	string	Nome DNS do nó (ex: "michel_1.farm.local")
role	string	Papel do nó no cluster ("master")
task	string	Tipo de relatório enviado ("performance_report")
timestamp	string (ISO-8601)	Momento da coleta no formato YYYY-MM-DDTHH:MM:SSZ
message_id	string (UUID)	Identificador único da mensagem
payload_version	string	Versão do schema do payload (ex: "sprint4-monitor-v2")

performance.system

Campo	Tipo	Descrição
uptime_seconds	int	Tempo de atividade do nó em segundos
load_average_1m	float	Média de load da CPU nos últimos 1 minuto
load_average_5m	float	Média de load da CPU nos últimos 5 minutos
cpu.usage_percent	float	Percentual de uso da CPU (0-100)

cpu.count_logical	int	Número de CPUs lógicas (threads)
cpu.count_physical	int	Número de CPUs físicas (cores)
memory.total_mb	int	Memória RAM total em MB
memory.available_mb	int	Memória RAM disponível em MB
memory.percent_used	float	Percentual de uso da memória (0–100)
memory.memory_used	int	Memória RAM utilizada em MB
disk.total_gb	float	Espaço em disco total em GB
disk.free_gb	float	Espaço em disco livre em GB
disk.percent_used	float	Percentual de uso do disco (0–100)

performance.farm_state.workers

Campo	Tipo	Descrição
total_registered	int	Total de workers atualmente registrados no nó
workers_utilization	int	Workers ocupados no momento (executando tarefas)
workers_alive	int	Workers considerados vivos/respondendo
workers_idle	int	Workers ociosos disponíveis para novas tarefas
workers_borrowed	int	Workers que este nó emprestou para outros nós
workers_received	int	Workers que este nó recebeu emprestados de outros nós

workers_failed	int	Workers que falharam
workers_home	int	Workers nativos do servidor (sem empréstimos)
workers_available_capacity	int	Capacidade ociosa total (= workers_idle)
borrowed_workers	array	Lista de workers emprestados com origem/destino

borrowed_workers[]

Campo	Tipo	Descrição
direction	string	Direção do empréstimo: "out" (emprestou para outro) ou "in" (recebeu de outro)
peer_uuid	string	server_uuid do nó na outra ponta do empréstimo

performance.farm_state.tasks

Campo	Tipo	Descrição
tasks_pending	int	Tarefas aguardando execução
tasks_running	int	Tarefas em execução no momento
tasks_completed	int	Total de tarefas concluídas
tasks_failed	int	Total de tarefas com falha
oldest_task_age_s	int	Idade da tarefa pendente mais antiga (segundos)

performance.config_thresholds

Campo	Tipo	Descrição
max_task	int	Número máximo de tarefas antes de considerar o nó saturado

warn_cpu_percent	int	Percentual de CPU para disparar alerta (ex: 85)
warn_memory_percent	int	Percentual de memória para disparar alerta (ex: 85)
release_task	int	Threshold para liberar workers emprestados

performance.neighbors[]

Campo	Tipo	Descrição
server_uuid	string	Identificador do nó vizinho
status	string	Status do vizinho: "available" ou "unavailable"
last_heartbeat	string (ISO-8601)	Timestamp do último heartbeat recebido do vizinho

Como enviar dados para o Supervisor

Para testar o seu projeto, você deve:

1. Enviar os dados de métricas em formato JSON, seguindo o template definido acima.
2. Abrir uma conexão TLS sobre TCP com o supervisor.
3. Enviar o JSON pela conexão estabelecida.
4. Não utilizar HTTP para esse envio.
5. Não aguardar resposta da aplicação após o envio. O cliente deve apenas conectar, enviar os dados e encerrar a conexão.

Parâmetros da conexão:

- Host: nuted-ia.dev
- Porta: 443
- Protocolo: TLS sobre TCP
- SNI: nuted-ia.dev

Exemplo de configuração no código:

```
TCP_SOCKET_HOST = "nuted-ia.dev"
TCP_SOCKET_PORT = 443
TCP_SOCKET_TLS = True
TCP_SOCKET_SNI = "nuted-ia.dev"
```

Importante:

- Não use caminhos como /supervisor/colector ou /supervisor no socket TCP.
- Em conexões TCP, o endpoint é definido apenas por host e porta.
- Não use bibliotecas HTTP para enviar o payload.
- O cliente deve apenas abrir a conexão, enviar o JSON e finalizar.
- Os servers não devem aguardar mensagem de retorno com recv.

Como visualizar o Dashboard

Todas as métricas enviadas pelos projetos são agregadas e exibidas em um painel visual interativo.

Para acompanhar o comportamento do cluster, acesse:

- URL do Dashboard: <https://nuted-ia.dev/supervisor/dashboard/>

Ao abrir esse endereço no navegador, você poderá:

- Visualizar a topologia dos nós, servers e workers;
- Acompanhar o consumo de CPU, memória, disco e filas de tarefas por servidor;
- Ver o estado dos workers, incluindo ativos, ociosos, emprestados e com falha;
- Identificar gargalos e nós sobrecarregados em tempo real.

Cada vez que seu projeto enviar novas métricas pela conexão TLS/TCP configurada, o dashboard será atualizado automaticamente, permitindo a validação e depuração do comportamento distribuído da sua aplicação.

Observação sobre os identificadores dos nós

Os valores michel_1 e michel_2 representam identificadores de farms em execução no ambiente do professor e devem ser usados no campo server_uuid do payload. Esses valores não fazem parte do endereço de conexão com o supervisor.

EXEMPLO DO DASHBOARD



SUPERVISOR // FARM

Prof. Michel Junio - Sistemas Distribuídos

2 Down / 1 High CPU

Alertas Ativos

00:54:19

CONTROLES DO CLUSTER

Timeout de Nó Morto (s)



Escala de Threshold



MASTERS ATIVOS
2

TAREFAS PENDENTES
143

TOTAL WORKERS
10

WORKERS EMPRESTADOS
4

CPU MÉDIA (%)
56.1

CPU MÁXIMA (%)
100.0

MEMÓRIA MÉDIA (%)
54.8

MÁX TAREFAS/NÓ
139

MEMÓRIA TOTAL (MB)
48918

CPUS LÓGICAS
32

DISCO TOTAL (GB)
1497.5

TOPOLOGIA DA REDE



STATUS WORKERS

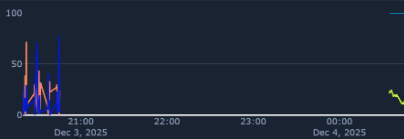


Ativos Emprestados
Recebidos Ociosos

LOG DE EVENTOS

[00:54:21] WARN: Nó michel_2 com CPU alta: 100.0%
[00:54:21] ERR: No SERVER_1 Down
[00:54:21] ERR: No SERVER_1.test Down

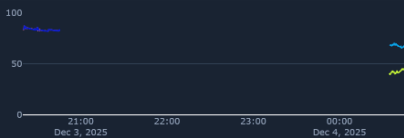
HISTÓRICO CPU



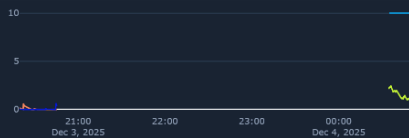
HISTÓRICO TAREFAS



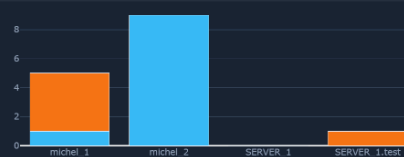
HISTÓRICO MEMÓRIA



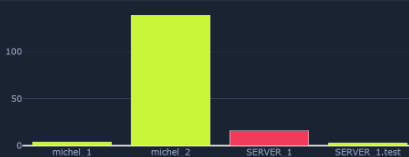
HISTÓRICO LOAD (1M)



WORKERS POR NÓ



TAREFAS POR NÓ (HOTSPOTS)



NÓS DA INFRAESTRUTURA (DETALHES)

michel_1 [UP]

Tarefas 4 CPU 12% MEM 42%

Task	Timestamp	Id Msg
performance_report	137.754	0b7394d

Tarefas P/R	CPU (U/P)	MEM %
4/1	12.2%	42.4%
(6/4)		

Workers TAU/AVG/R/T
1/1/1/0/4/3/0

Uptime	Load 1m/5m
3h	1.22/2.50

Memória (MB)
Tot:16384 / Disp:8192 / Use:8000

Disco (GB)
Tot:512.0 / Liv:250.0 / 45.0%

Threshold	Last Seen	Válidos
150.0	00:54:15	1

michel_2 [UP]

Tarefas 139 CPU 100% MEM 67%

Task	Timestamp	Id Msg
performance_report	141.155	5946680

Tarefas P/R	CPU (U/P)	MEM %
139/9	100.0%	67.5%
(6/4)		

Workers TAU/AVG/R/T
9/9/9/0/0/4/0

Uptime	Load 1m/5m
3h	10.00/2.50

Memória (MB)
Tot:16384 / Disp:8192 / Use:8000

Disco (GB)
Tot:512.0 / Liv:250.0 / 45.0%

Threshold	Last Seen	Válidos
150.0	00:54:05	1

SERVER_1 [DOWN]

Tarefas 16 CPU 24% MEM 83%

Task	Timestamp	Id Msg
performance_report	123.419	026d84e

Tarefas P/R	CPU (U/P)	MEM %
16/0	24.0%	82.0%
(6/4)		

Workers TAU/AVG/R/T
1/0/1/0/0/0/0

Uptime	Load 1m/5m
0h	0.00/0.00

Memória (MB)
Tot:8095 / Disp:1389 / Use:6686

Disco (GB)
Tot:236.8 / Liv:166.2 / 29.0%

Threshold	Last Seen	Válidos
15.0	20:45:23	1

SERVER_1.test [DOWN]

Tarefas 3 CPU 4% MEM 83%

Task	Timestamp	Id Msg
performance_report	122.311	a5651d64

Tarefas P/R	CPU (U/P)	MEM %
3/0	4.2%	82.0%
(6/4)		

Workers TAU/AVG/R/T
2/0/1/0/0/0/0

Uptime	Load 1m/5m
0h	0.54/0.13

Memória (MB)
Tot:8075 / Disp:1386 / Use:6689

Disco (GB)
Tot:236.8 / Liv:166.2 / 29.0%

Threshold	Last Seen	Válidos
15.0	20:45:22	1